

#1

ForensicSculptor – Offline AI-Driven Digital Evidence Carver

Problem Statement

Digital investigators often struggle to recover deleted or fragmented files from seized storage media without specialised commercial tools that require licences, internet validation, or cloud-based processing. In field conditions with no connectivity, critical evidence can be lost or delayed, compromising legal cases.

Proposed Solution

A desktop Java application that reads raw disk images or attached drives and uses intelligent file carving algorithms enhanced by offline machine learning. It reconstructs files by recognising file headers, footers, and internal structures, even when file system metadata is destroyed. A local deep-learning model classifies file fragments to stitch together JPEGs, PDFs, Office documents, etc., with high accuracy.

Innovation Factor

- First open-source, fully offline carving tool with embedded neural fragment classification
- Uses a custom-trained model (exported to Deeplearning4j) that runs entirely on the local CPU, no GPU needed
- Advanced validation via file-specific entropy and structure verification beyond simple magic bytes
- Designed for field forensics with a streamlined, keyboard-driven interface suitable for high-pressure environments

Abstract (310 words)

Digital forensics practitioners face an ever-growing volume of data on seized devices, often with intentionally damaged file systems. File carving is the last resort to recover evidence, yet existing open-source tools either lack intelligence—relying solely on header/footer signatures—or depend on cloud services for deep analysis.

ForensicSculptor bridges this gap with a desktop application that performs advanced, AI-assisted carving completely offline.

The system accepts raw disk images or physical drives via direct block-level access (using JNA to call native APIs on Windows/Linux) and first performs a high-speed signature-based scan for intact files. For unallocated space and fragments, it invokes a byte-level recurrent neural network trained on millions of file fragments to classify the

data type (JPEG, PNG, PDF, ZIP, DOCX, etc.) and predict fragment boundaries. The model was pre-trained offline and serialised into a Deeplearning4j model file bundled with the application. A graph-based reassembly algorithm then sequences the classified fragments by comparing edge compatibility scores, effectively reconstructing files that traditional carvers miss.

The user interface, built with JavaFX, provides a timeline view of recovered artifacts, a hex preview, and integrated metadata extraction. All processing is logged in a forensically sound manner, preserving chain-of-custody records. The tool includes a local file-signature database and a plugin architecture for custom carving rules. By removing any dependency on internet access or commercial licences, ForensicSculptor empowers law enforcement, military, and corporate investigators in disconnected environments, drastically reducing evidence recovery time and cost. The project demonstrates a unique combination of low-level system programming, offline AI, and domain-specific forensic science, making it a standout candidate for national-level competition.

Core Features

- Raw disk/image ingestion with block-level read (NTFS, FAT, ext, etc.)
- Signature-based file carving for intact files
- AI-powered fragment classification using an offline Deeplearning4j RNN
- Graph-based reassembly of fragmented files
- Built-in file viewer (hex, image, text previews)
- Export recovered files with metadata report
- Forensic logging and chain-of-custody export
- Plugin system for custom carving signatures

Functional Requirements

1. Load raw image files (dd, E01, raw) and physical drives.
2. Detect partition tables and automatically extract file system metadata for guided carving.
3. Perform header/footer carving for 30+ file types.
4. Train and embed a pre-trained neural network classifier for fragment type identification.
5. Allow manual carving rule addition via XML/JSON.
6. Visualise disk allocation map and highlight recovered files.
7. Generate PDF/HTML evidence report with hashes and timestamps.

Non-Functional Requirements

- Performance: Carve a 500 GB image within 4 hours on a quad-core laptop.
- Security: Operate with least privilege; never modify source evidence.
- Reliability: Resume interrupted carving sessions from checkpoint files.
- Usability: Intuitive dark-theme forensic interface with single-click actions.

Suggested Technologies

- JavaFX (UI)
- JNA for raw disk access
- Deeplearning4j (neural network inference)
- SQLite (local case DB)
- Apache Tika (metadata extraction)
- Java Native Runtime for drive enumeration

System Modules

1. **Disk Acquisition Module** – enumerates devices, reads sectors.
2. **Signature Engine** – matches byte patterns.
3. **Fragment Classifier** – DL4J model inference.
4. **Reassembly Engine** – graph construction and file reconstruction.
5. **Evidence Manager** – case database, logging, export.
6. **Viewer Module** – hex/text/image display.

Architecture Recommendation

Modular MVC with a pipeline architecture: Acquisition → Carving Pipeline (Signature + Classifier → Assembler) → Evidence Store → Viewer. Loose coupling via events.

Database Design Overview

Tables: Case, Device, Partition, CarvedFile, Fragment, RecoveryLog. Entity–attribute design to preserve forensic integrity.

AI/ML Possibility

Yes – offline neural fragment classifier (RNN/CNN) using Deeplearning4j; model trained externally and bundled.

Difficulty Level

Advanced

Estimated Completion Time

8 weeks (2 weeks architecture + disk I/O, 3 weeks carving & ML integration, 2 weeks reassembly, 1 week UI/reporting)

Competition Winning Potential

Extremely high – addresses a genuine law enforcement need with a novel AI approach, demonstrates deep systems programming and ML, and is fully functional offline. Judges will appreciate the social impact and technical depth.

Future Scope

- Add video file reconstruction
 - Integrate facial recognition on recovered images (offline)
 - Extend to mobile flash memory chips via JTAG/SD card reader support
-

#2

PlagiarEyes – AST-Based Code Plagiarism Detective

Problem Statement

Academic institutions face rampant source-code plagiarism, yet existing detection tools either rely on simple text similarity (easily fooled by variable renaming) or are cloud-based subscription services that compromise student code privacy.

Proposed Solution

An offline desktop application that parses source code into Abstract Syntax Trees (ASTs) using language-specific parsers (JavaParser for Java, ANTLR for Python/C++) and computes structural similarity using tree edit distance and sub-tree hashing. It detects logical plagiarism even after heavy obfuscation, while keeping all submissions strictly on-premises.

Innovation Factor

- Deep structural comparison via AST fingerprinting, resistant to superficial changes
- Clustering algorithm to group suspicious submissions and generate a visual similarity network graph
- Fully offline, preserving intellectual privacy and academic integrity
- User-friendly batch submission interface, generating detailed side-by-side diff reports of suspicious pairs

Abstract (290 words)

Source code plagiarism threatens the credibility of programming courses, but commercial detectors often transmit student code to external servers, raising privacy concerns. PlagiarEyes is a Java desktop solution that performs intelligent plagiarism analysis entirely offline. It accepts a directory of student submissions, identifies the programming language, parses each file into an AST, normalises identifier names, and then computes structural similarity metrics. The core algorithm combines tree edit distance with locality-sensitive hashing of AST subtrees to quickly identify near-duplicates and partially copied logic.

The application, built with JavaFX, provides a rich visualisation of the submission corpus as a similarity network, where nodes represent students and edges denote high similarity. An investigator can click an edge to see a side-by-side comparison view, with AST nodes colour-coded to reveal matching structures. A built-in report generator produces evidence packages for academic committees. PlagiarEyes supports Java, Python, C, and C++ out of the box, with an extension API for additional languages.

By operating offline, the tool ensures that sensitive student work never leaves the evaluator's computer. The use of AST analysis makes it exceptionally robust against common evasion techniques like renaming, loop reordering, or comment insertion. This project uniquely blends compiler theory, graph algorithms, and visual analytics into a practical, privacy-respecting academic integrity tool.

Core Features

- Multi-language parser support (Java, Python, C/C++)
- AST generation and normalization (identifier abstraction)
- Sub-tree hashing and locality-sensitive hashing for fast similarity search
- Tree edit distance computation for detailed pair scoring
- Interactive similarity network graph (force-directed layout)
- Side-by-side diff view with AST highlighting
- Batch submission processing and customisable thresholds
- PDF evidence report generation

Functional Requirements

1. Load assignment directory and auto-detect language.
2. Parse each file and cache ASTs in local DB for efficiency.
3. Compute pairwise similarity matrix using configurable mix of hashing and edit distance.
4. Cluster submissions and flag pairs above a threshold.
5. Visualise network and allow drilling into comparisons.

6. Export reports with matched code snippets.

Non-Functional Requirements

- Performance: Process 500 submissions (average 200 LOC each) under 2 minutes.
- Accuracy: >95% recall on obfuscated plagiarism test set.
- Usability: Simple drag-and-drop folder interface.

Suggested Technologies

- JavaFX
- JavaParser, ANTLR 4 for parsing
- SQLite (submission metadata and cached ASTs)
- JGraphT (graph algorithms)
- GraphStream or custom JavaFX canvas for network visualisation

System Modules

1. **Language Adapter** – parsing strategies
2. **AST Normalizer** – abstracts identifiers, removes comments
3. **Similarity Engine** – hashing + tree edit distance
4. **Clustering & Visualisation**
5. **Report Generator**

Architecture Recommendation

Pipeline with adapter pattern for language parsers; use a local embedded database to store serialised ASTs for incremental re-runs.

Database Design Overview

Submission, ASTCache, SimilarityScore, Cluster. ASTs stored as blobs.

AI/ML Possibility

Optional – could use a small offline classifier (Weka) to learn from instructor feedback and adjust similarity thresholds.

Difficulty Level

Intermediate

Estimated Completion Time

8 weeks (3 weeks parsing infra, 3 weeks similarity algorithms, 2 weeks visualisation)

Competition Winning Potential

Strong – solves a pervasive, real-world problem with technical elegance and privacy focus; visual network makes it demonstration-friendly.

Future Scope

- Add natural language comment analysis
 - Cross-language plagiarism detection
 - Integration with LMS systems via export formats
-

#3

RescueMap Commander – Offline Disaster Coordination Simulator

Problem Statement

During natural disasters, internet infrastructure is often destroyed, yet emergency operations centres need geospatial intelligence, resource tracking, and logistic planning. Paper maps and radios remain fallback, leading to inefficient coordination and loss of life.

Proposed Solution

A desktop application that uses offline OpenStreetMap vector tiles (bundled) to provide a full-featured GIS for disaster management. It simulates disaster impact using an offline agent-based model, tracks resources (shelters, medical supplies, vehicles), and optimises allocation via constraint solving – all without any network.

Innovation Factor

- Fully offline map engine using Mapsforge with vector rendering
- Built-in disaster simulation (flood, earthquake) with configurable parameters, visually unfolding on the map
- Resource allocation optimiser powered by a local OR-Tools-style constraint solver (Choco-solver)
- Realistic scenario mode where students can run “Incident Command” exercises with time pressure

Abstract (350 words)

Emergency response in the immediate aftermath of a catastrophe is often hamstrung by the collapse of communication networks. RescueMap Commander fills this gap by providing an offline desktop command-and-control tool that can be pre-loaded with region-specific map data and run on a ruggedised laptop in the field. The system uses the Mapsforge library to render high-quality OpenStreetMap vector maps without internet, supporting zoom, pan, and layer toggling. The map is augmented with custom overlays for critical infrastructure (hospitals, fire stations), hazard zones, and dynamic resource markers.

A unique offline simulation engine, built on agent-based modelling, allows commanders to inject a disaster scenario (e.g., an earthquake of magnitude X at location Y) and watch the propagation of damage, affected population estimates, and road blockages on the map. This is not just a static display but an active model that updates in real time as response actions are taken. The resource management module employs a constraint programming library (Choco) to suggest optimal deployment of limited ambulances, medical teams, and supplies, minimising response time while respecting capacity constraints.

The interface, built with JavaFX, offers a dual-pane view: a map canvas with a collapsible dashboard for incident logs, resource status, and allocation plans. All data—map tiles, demographic databases, and simulation parameters—is stored locally in a combination of SQLite and spatial index files. A scenario editor allows emergency planners to pre-configure exercises, which can then be used for training or live response. RescueMap Commander demonstrates how a sophisticated, AI-infused geospatial tool can be built for zero-connectivity environments, combining GIS, simulation, and optimisation in one cohesive desktop system.

Core Features

- Offline vector map rendering with Mapsforge
- Disaster simulation engine (earthquake, flood, fire) with visual impact propagation
- Resource management (vehicles, personnel, supplies) with CRUD and status tracking
- Optimal allocation solver (Choco constraint solver)
- Incident logging and timeline
- Scenario editor for training exercises
- Export of operation plans as PDF

Functional Requirements

1. Load pre-downloaded map regions from local storage.

2. Create/load disaster scenario with parameters (type, epicentre, intensity).
3. Visualise damage zones and affected population dynamically.
4. Add and manage resources with attributes (type, capacity, location).
5. Run solver to allocate resources to incidents.
6. Manual override and drag-drop reassignment on map.
7. Maintain a chronological incident log.

Non-Functional Requirements

- Performance: Map rendering <100 ms per frame, solver under 5 seconds for 50 resources, 200 incidents.
- Usability: Intuitive map interaction, familiar GIS paradigms.
- Reliability: Crash recovery autosave.

Suggested Technologies

- JavaFX with Mapsforge for offline maps
- Choco-solver (constraint programming)
- SQLite for resource/incident DB
- GeoTools (for spatial operations, if needed offline)
- JTS Topology Suite

System Modules

1. **Map Engine** – tile loading, rendering, overlay management
2. **Simulation Core** – agent-based model engine
3. **Resource Manager** – CRUD and status
4. **Allocation Optimiser**
5. **Scenario Editor**
6. **Report Module**

Architecture Recommendation

Layered architecture: Presentation (JavaFX map+panels) → Domain (simulation, optimisation) → Infrastructure (spatial DB, file storage). Use observer pattern to update map from simulation.

Database Design Overview

Tables: Scenario, Incident, Resource, Allocation, MapRegion. Spatial data stored as WKT blobs.

AI/ML Possibility

Yes – the simulation engine can be enhanced with a simple offline ML model to predict resource demand based on disaster parameters (using Weka regression).

Difficulty Level

Advanced

Estimated Completion Time

8 weeks (2 weeks map setup, 3 weeks simulation & solver, 2 weeks UI, 1 week scenario editor)

Competition Winning Potential

Exceptional – directly addresses a life-saving domain with a complete, functional, and visually compelling offline system. The blend of GIS, simulation, and optimisation will impress judges.

Future Scope

- Multi-laptop peer-to-peer sync over local ad-hoc Wi-Fi (offline mesh)
 - Drone footage overlay for damage assessment
 - Integration with offline weather prediction models
-

#4**MediVault Analyzer – Privacy-First Personal Health Record Interpreter****Problem Statement**

Patients accumulate scattered medical reports (PDF scans, lab printouts), but interpreting them requires medical knowledge. Existing health apps demand cloud uploads, violating privacy. There is no offline tool that extracts, analyses, and trends personal health data while keeping it strictly local.

Proposed Solution

A Java desktop application that ingests scanned medical documents via Tesseract OCR (offline) and structured PDFs, extracts numerical lab results, medication names, and diagnoses using NLP and pattern matching. It builds a personal health timeline, applies

offline ML anomaly detection, and checks for dangerous drug interactions using a bundled open-source drug database.

Innovation Factor

- Complete offline processing of unstructured medical documents with OCR and NLP
- Offline anomaly detection on lab trends using a Weka-based time-series model
- Local drug-drug interaction checker using a parsed subset of DrugBank (open data, no live look-up)
- Privacy-centric design: all data encrypted at rest, zero-network, user-owned

Abstract (330 words)

Managing personal health is challenging when medical history is fragmented across PDFs, images, and paper. MediVault Analyzer transforms any laptop into a secure, offline health data analyst. The user scans or drag-drops a medical report; the application first performs OCR using Tesseract (bundled with language data) to extract text, then applies a specialised NLP pipeline to identify structured entities: lab test names, numeric values, reference ranges, dates, and medication names. This pipeline uses a rule-based engine augmented with a small offline dictionary of common medical terms.

Extracted data is stored in a local, encrypted SQLite database. The system automatically plots time-series graphs for each lab parameter, overlaying normal reference bands. A Weka-trained anomaly detector (Isolation Forest or One-Class SVM, serialised) flags values that deviate from personal or population baselines, alerting the user to potential health issues. The medication module cross-references active ingredients against a locally stored drug interaction table to warn of adverse combinations. All analysis happens without any internet connection, guaranteeing absolute privacy.

The JavaFX interface is designed for non-technical users, featuring a “health dashboard” with colour-coded trends and a medication schedule view. Reports can be exported as a comprehensive medical summary for sharing with a doctor (with patient consent). MediVault Analyzer exemplifies how advanced data extraction and machine learning can be brought offline to empower individuals in a deeply sensitive domain, making it a strong contender for competitions focusing on healthcare and privacy innovation.

Core Features

- OCR ingestion of scanned reports (Tesseract)
- NLP-based entity extraction (lab values, medications)
- Personal health trend visualisation with interactive charts
- Offline anomaly detection on lab results
- Drug interaction checker (local database)

- Encrypted local storage
- PDF report generation for physician visits

Functional Requirements

1. Import image/PDF files and execute OCR.
2. Parse text to extract structured health data.
3. Store data in encrypted SQLite with user-defined master password.
4. Plot multi-parameter time series with custom date ranges.
5. Run anomaly detection model and highlight abnormal results.
6. Manually add health data entries.
7. Check for drug interactions among current medications.

Non-Functional Requirements

- Security: AES-256 encryption for DB, master password hashing.
- Performance: OCR on a 3-page report < 15 seconds.
- Usability: Accessible UI with large fonts, simple navigation.

Suggested Technologies

- JavaFX with charts library
- Tesseract OCR (JavaCPP-Presets)
- Apache OpenNLP or Stanford CoreNLP (offline models) for medical NER
- SQLite with SQLCipher (for encryption)
- Weka for anomaly detection model
- iText or Apache PDFBox for PDF generation

System Modules

1. **Document Ingestion** – PDF/Image import and OCR
2. **Medical NLP Pipeline** – entity extraction
3. **Health Data Store** – encrypted repository
4. **Analytics Engine** – trend charts, anomaly detection
5. **Drug Interaction Checker**
6. **Report Builder**

Architecture Recommendation

Clean Architecture with clear separation: UI → Application Core (NLP, Anomaly) → Domain (Patient, LabRecord) → Infrastructure (DB, OCR). Use dependency injection.

Database Design Overview

Tables: PatientProfile, LabResult, Medication, Allergy, Observation. Encrypted file.

AI/ML Possibility

Yes – offline anomaly detection using Weka's Isolation Forest trained on synthetic lab data; model bundled as file.

Difficulty Level

Intermediate–Advanced

Estimated Completion Time

8 weeks (2 weeks OCR & parsing, 3 weeks NLP & extraction, 2 weeks analytics & UI, 1 week encryption & packaging)

Competition Winning Potential

Very high – addresses the sensitive intersection of health data privacy and AI, with a tangible self-contained product that judges can immediately see value in.

Future Scope

- Add dietary and lifestyle logging with correlation analysis
 - Integrate Bluetooth health device data import (offline)
 - Federated learning across family members (local network only)
-

#5

VoiceLink AAC – Intelligent Offline Communication Aid for Non-Verbal Individuals

Problem Statement

People with speech impairments often rely on Augmentative and Alternative Communication (AAC) devices. Commercial solutions are expensive, require cloud services for advanced prediction, and many fail in areas without internet, leaving users isolated.

Proposed Solution

A desktop Java application that serves as a full-featured AAC system. It offers

symbol-based grid communication, predictive text with an offline language model, and a webcam-based eye-tracking/gaze selection module for users with severe motor disabilities. Text-to-speech output uses FreeTTS, all offline.

Innovation Factor

- Combines symbol grids, keyboard prediction, and gaze-based access in one offline suite
- Eye-tracking via ordinary webcam using OpenCV and a lightweight offline pupil detection algorithm, no dedicated hardware
- Adaptive prediction that learns user's phrase patterns over time (simple Markov model)
- Fully configurable communication boards, enabling personalization without internet

Abstract (340 words)

AAC devices are lifelines for millions, yet affordability and internet dependency create barriers. VoiceLink AAC is a free, offline, Java-based desktop application that transforms any laptop into a powerful communication tool. Its core interface is a customisable grid of symbols and text cells derived from open symbol sets (like Mulberry Symbols). Users can compose messages by clicking or dwelling on cells; the message is spoken aloud via FreeTTS, a high-quality offline speech synthesizer.

For individuals with limited motor control, VoiceLink incorporates a "Gaze Mode" using the built-in webcam. A lightweight pupil-detection algorithm (based on OpenCV's Haar cascades and morphological operations) tracks the user's eye movements and maps gaze coordinates to screen regions. A dwell-time mechanism triggers selection, making the system accessible without any external hardware. The prediction engine uses an n-gram language model trained on general text and gradually adapted to the user's own vocabulary, stored entirely in a local SQLite database. The prediction significantly speeds up communication by suggesting next words or full phrases.

VoiceLink AAC includes a Board Editor that lets caregivers or speech therapists create new symbol grids, import images, and link cells to phrases or actions (e.g., playing a sound). All configuration is saved locally. The application runs on standard laptops, including low-cost machines, and requires no internet. It addresses a clear social inclusion gap by providing a feature-rich, offline AAC solution that can be deployed in rural clinics, special education classrooms, or emergency situations where connectivity is unavailable.

Core Features

- Symbol-grid based communication interface
- Text-to-speech with FreeTTS (configurable voice)
- Predictive text using offline language model (n-gram + personalisation)

- Gaze selection mode via webcam (pupil tracking)
- Board editor for creating custom communication layouts
- Phrase history and favourites
- Multi-lingual support (bundled symbol sets)

Functional Requirements

1. Load and navigate symbol boards (hierarchical).
2. Compose message and speak it.
3. Auto-predict next words based on typed/symbol input.
4. Activate gaze mode, calibrate, and select cells by dwelling.
5. Create/Edit boards with drag-and-drop cell placement.
6. Save user profiles and learned prediction data.

Non-Functional Requirements

- Performance: Voice response < 200 ms; gaze tracking at 15 fps on integrated webcam.
- Usability: Large touch-friendly targets, high-contrast themes for visual impairment.
- Reliability: Robust to lighting changes with adaptive thresholding.

Suggested Technologies

- JavaFX (UI)
- FreeTTS (text-to-speech)
- OpenCV (JavaCV) for webcam capture and image processing
- SQLite (user data and language model)
- Apache Commons Math (n-gram probability)

System Modules

1. **Communication Interface** – grid rendering, input handling
2. **Speech Engine** – FreeTTS integration and voice management
3. **Prediction Engine** – language model and personalisation
4. **Gaze Tracker** – webcam capture, pupil detection, calibration
5. **Board Editor**
6. **Profile Manager**

Architecture Recommendation

Model-View-Controller (MVC) with JavaFX. The gaze tracker runs in a separate thread, publishing events to the UI.

Database Design Overview

Tables: UserProfile, Board, Cell, PhraseHistory, NGramModel. Lightweight.

AI/ML Possibility

Yes – online learning of user phrase patterns; eye-tracking uses classical CV, not deep learning, to stay lightweight.

Difficulty Level

Intermediate–Advanced

Estimated Completion Time

8 weeks (2 weeks basic AAC interface, 2 weeks TTS & prediction, 2 weeks gaze tracking, 2 weeks board editor & testing)

Competition Winning Potential

High social impact; judges favour assistive technology. The multi-modal access (touch, mouse, gaze) and fully offline nature make it distinct.

Future Scope

- Switch scanning for single-switch users
 - Bluetooth switch interface
 - Emotion detection for adaptive communication suggestions
-

#6

SignFlow Tutor – Real-Time Offline Sign Language Practice with Feedback

Problem Statement

Learning sign language requires interactive practice and feedback, but existing tools either are passive video libraries or cloud-based AI that requires internet, leaving learners in low-connectivity regions without affordable, private tutoring.

Proposed Solution

A desktop app that uses the webcam to capture the learner's hand gestures, runs an offline hand-pose estimator, and compares it to a target sign. It provides real-time

corrective feedback (e.g., “move right hand higher”) and tracks learning progress, all without internet.

Innovation Factor

- Fully offline hand-pose estimation using a custom lightweight CNN exported to Deeplearning4j (trained on synthetic data) or classical CV with contour analysis and Hu moments
- Real-time feedback via skeleton overlay and textual hints
- Curriculum of signs structured by difficulty
- Gamified learning with scoring and streak tracking

Abstract (320 words)

Sign language is a critical communication bridge for the Deaf community, but learning it effectively demands interactive practice. SignFlow Tutor brings an intelligent, offline practice environment to any laptop. When a learner selects a sign from the built-in dictionary, the application displays a reference video and then activates the webcam to capture the user’s attempt. The system extracts hand region and pose features using either a lightweight pre-trained neural network (Deeplearning4j CNN on hand keypoints) or a classical computer vision pipeline (skin colour segmentation, convexity defects, finger counting). It then computes a similarity score between the performed gesture and the target sign’s feature vector, providing immediate visual feedback: a colour-coded skeleton overlay shows which hand parts are misaligned, and textual cues suggest corrections (“bend index finger”, “raise palm”).

All processing runs on the CPU, maintaining 10+ frames per second on a standard laptop. The built-in curriculum organises signs into lessons, and a spaced-repetition algorithm ensures efficient memorisation. Learner progress, including accuracy history and practice time, is stored locally. The system includes an authoring tool so instructors can add new signs by recording reference videos and manually annotating key poses. SignFlow Tutor democratises sign language education by eliminating the need for internet or expensive motion-capture hardware, making it ideal for rural schools and self-learners.

Core Features

- Webcam-based gesture capture and analysis
- Offline hand-pose recognition (CV + ML hybrid)
- Real-time skeleton overlay and feedback
- Structured curriculum with lessons and quizzes
- Progress tracking and spaced repetition

- Sign authoring tool for custom content
- Visual scoring and gamification

Functional Requirements

1. Present target sign as video/image and prompt user to perform.
2. Capture webcam feed and detect hand region.
3. Extract pose features and compare to reference.
4. Provide real-time feedback overlay and text correction.
5. Log session data and compute proficiency scores.
6. Allow creation of new signs with annotation.

Non-Functional Requirements

- Performance: ≥ 10 fps processing on i5 laptop.
- Usability: Simple calibration and easy-to-follow feedback.
- Reliability: Works under varied lighting with dynamic threshold adjustment.

Suggested Technologies

- JavaFX (UI and webcam display)
- JavaCV/OpenCV for image processing
- Deeplearning4j (optional, for CNN keypoint model) or use classical CV moments
- SQLite (progress data)
- FreeTTS (audio cues)

System Modules

1. **Curriculum Manager** – lesson structure
2. **Hand Detector** – segmentation and feature extraction
3. **Gesture Comparator** – similarity scoring
4. **Feedback Renderer** – overlay and text
5. **Authoring Tool**
6. **Progress Tracker**

Architecture Recommendation

MVP with reactive stream from camera. Use background thread for CV, update JavaFX UI via Platform.runLater.

Database Design Overview

Tables: User, Sign, Lesson, Attempt, Proficiency.

AI/ML Possibility

Yes – offline CNN for hand keypoints; or classical CV with ML classifier (Weka) for gesture recognition.

Difficulty Level

Intermediate–Advanced

Estimated Completion Time

8 weeks (2 weeks CV pipeline, 3 weeks gesture matching & feedback, 2 weeks curriculum/UI, 1 week authoring)

Competition Winning Potential

High – innovative use of offline CV for education and inclusivity; live demo with webcam is compelling for judges.

Future Scope

- Two-way sign-to-text translation (interpret a performed sign)
 - Support for multiple sign languages
 - Collaborative local network practice sessions
-

#7**CogniCare Suite – Adaptive Cognitive Training & Early Dementia Risk Indicator****Problem Statement**

Cognitive decline in the elderly often goes undetected until severe. Existing brain-training apps are generic, require internet, and do not provide clinical insights. There is a need for a privacy-respecting, offline tool that adaptively trains cognitive functions and flags early warning signs.

Proposed Solution

A desktop application offering a set of scientifically inspired cognitive games (memory, attention, executive function) that dynamically adjust difficulty based on user performance. It applies an offline ML classifier to trajectory patterns to detect potential mild cognitive impairment, generating a report for healthcare providers—all local.

Innovation Factor

- Adaptive difficulty algorithm using Item Response Theory (IRT) principles
- Local ML model (Weka Random Forest) trained on cognitive performance datasets to estimate risk
- Multi-modal games (remembering sequences, Stroop test, spatial navigation) integrated in a unified suite
- Designed with elderly-friendly UI: large fonts, high contrast, simple navigation

Abstract (330 words)

Early detection of cognitive impairment can significantly improve management, yet regular screening is inaccessible in many communities. CogniCare Suite is a desktop-based cognitive training and assessment platform that operates entirely offline, making it suitable for home use or rural clinics. It comprises a battery of engaging games targeting specific cognitive domains: working memory (n-back, pattern recall), processing speed (Stroop task), executive function (Tower of Hanoi variant), and visuospatial ability (mental rotation). The difficulty of each task adapts in real time using a probabilistic model that estimates the user's current ability level, ensuring the challenge remains within an optimal zone for neuroplasticity.

The system records detailed performance metrics (reaction time, accuracy, learning rate) and stores them in a local SQLite database. After a series of sessions, an offline machine learning model (a Random Forest classifier serialised from Weka) analyses the longitudinal data to compute a cognitive risk score. This model was pre-trained on publicly available cognitive assessment datasets and fine-tuned for in-app use. The output is a private report that can be shared with a physician, highlighting trends and flagging anomalies.

The user interface is specifically crafted for older adults: large touch-friendly buttons, clear instructions with audio prompts (FreeTTS), and a distraction-free layout. Caregivers can also configure daily reminder pop-ups. CogniCare Suite demonstrates a powerful combination of health technology, adaptive learning, and offline AI, addressing a pressing social challenge with a compassionate, data-driven tool.

Core Features

- Adaptive cognitive games (memory, attention, executive function)
- Real-time difficulty adjustment
- Longitudinal performance dashboard
- Offline ML-based risk assessment model
- Audio instructions and feedback

- Elderly-optimised UI
- Caregiver report generator

Functional Requirements

1. Provide a selection of cognitive games with adaptive difficulty.
2. Collect detailed per-trial metrics (time, errors).
3. Adjust game parameters after each round based on IRT.
4. Store session data and compute trends.
5. Execute the risk classifier on session history and produce a score.
6. Generate a printable cognitive report.

Non-Functional Requirements

- Usability: WCAG-inspired accessibility; works with screen readers.
- Performance: Game response < 50 ms.
- Security: All health data encrypted at rest.

Suggested Technologies

- JavaFX (UI)
- FreeTTS (audio)
- SQLite with optional SQLCipher
- Weka (Random Forest classifier)
- Java 2D for simple game graphics

System Modules

1. **Game Engine** – game implementations and adaptive controller
2. **Session Recorder** – data capture
3. **Adaptive Logic** – IRT-based parameter tuning
4. **Risk Assessor** – ML classifier integration
5. **Report Generator**
6. **Caregiver Dashboard**

Architecture Recommendation

Modular game architecture: each game as a plugin conforming to a standard interface.
Shared scoring service and adaptive controller.

Database Design Overview

Tables: User, GameSession, TrialResult, RiskAssessment.

AI/ML Possibility

Yes – offline cognitive risk classifier with Weka; adaptive algorithm itself is a form of AI.

Difficulty Level

Intermediate

Estimated Completion Time

8 weeks (3 weeks games & adaptive logic, 2 weeks ML integration, 2 weeks UI/accessibility, 1 week reporting)

Competition Winning Potential

High – combines healthcare, AI, and accessibility in a socially impactful project; judges appreciate the human-centred design.

Future Scope

- Add physiological sensor support (heart rate via Bluetooth)
 - Multi-user family mode with comparative insights
 - Integration with open-source electronic health records
-

#8**BioGuard Console – Offline Biometric Access Control Manager****Problem Statement**

Small-scale secure facilities (research labs, server rooms) need biometric access control but commercial systems are expensive, network-dependent, and often lock users into proprietary ecosystems.

Proposed Solution

A Java desktop application that manages a local biometric authentication system using fingerprint and face recognition. It interfaces with low-cost USB fingerprint readers (using the open-source SourceAFIS library) and a webcam for face recognition via eigenfaces or LBPH, all offline. It controls an electronic door latch via a simple USB relay board.

Innovation Factor

- Fully offline multi-modal biometric management console
- Fingerprint matching with SourceAFIS (pure Java, no native drivers beyond USB HID) and face recognition with OpenCV's LBPH (trained models stored locally)
- Role-based access with time-based scheduling (e.g., lab open only 9-5)
- Physical device integration using serial/relay control via Java libraries (jSerialComm), completely air-gapped

Abstract (300 words)

Many small organisations need robust physical access control but cannot afford complex, networked systems. BioGuard Console is a self-contained, offline Java desktop application that turns a standard PC into a biometric access management panel. It supports two biometric modalities: fingerprint and face. Fingerprint enrollment and matching are handled by SourceAFIS, a pure Java algorithm that extracts minutiae from USB fingerprint scanner images (the scanner is treated as a standard image input device). Face recognition employs OpenCV's Local Binary Patterns Histograms (LBPH) recognizer, trained on user face images captured during enrollment.

The application maintains a local user database (SQLite encrypted) storing biometric templates, not raw images, ensuring privacy. Administrators can define roles and access schedules for each door/zone. When a user presents a fingerprint or looks at the webcam, the system verifies their identity against stored templates and, if authorised within the current time window, triggers a USB relay board (via jSerialComm) to unlock the door. All events are logged with timestamps for audit.

The JavaFX interface provides a dashboard showing recent access events, system status, and quick enrollment wizards. The entire system is designed to operate in an air-gapped environment—no network interface is ever used. BioGuard Console offers a unique blend of biometrics, hardware control, and security, packaged as a practical, low-cost solution that students can fully demonstrate with a physical prototype.

Core Features

- Fingerprint enrollment and 1:N matching (SourceAFIS)
- Face enrollment and recognition (OpenCV LBPH)
- Multi-modal fusion (optional, require both)
- Role-based access control with time scheduling
- USB relay/door strike control
- Audit log viewer
- Admin dashboard

Functional Requirements

1. Enroll user with fingerprint (multiple samples) and face images.
2. Store biometric templates securely.
3. Match live fingerprint/face against database.
4. Evaluate access rules (time, role, door).
5. Activate relay for a configurable duration on success.
6. Log all attempts and generate reports.

Non-Functional Requirements

- Security: Biometric templates encrypted; match threshold tunable.
- Performance: 1:100 fingerprint match < 1 second.
- Reliability: Graceful fallback if camera/fingerprint scanner unavailable.

Suggested Technologies

- JavaFX
- SourceAFIS (fingerprint matching)
- OpenCV via JavaCV (face recognition)
- SQLite with encryption (SQLCipher)
- jSerialComm (USB relay control)
- JNA for USB device detection

System Modules

1. **Biometric Core** – fingerprint engine, face engine
2. **Access Controller** – rule evaluator and relay trigger
3. **User Manager** – enrollment, role assignment
4. **Audit Logger**
5. **Admin Dashboard**

Architecture Recommendation

Layered: UI → Biometric Service → Access Decision Engine → Hardware Abstraction Layer → Relay Driver.

Database Design Overview

Tables: User, FingerprintTemplate, FaceModel, AccessRule, AccessLog.

AI/ML Possibility

Yes – face recognition using LBPH (a classic ML model) trained offline.

Difficulty Level

Intermediate

Estimated Completion Time

8 weeks (2 weeks biometric libraries integration, 3 weeks access logic & UI, 2 weeks hardware control, 1 week testing)

Competition Winning Potential

Very strong – physical computing demonstration with biometrics and real-world security application; judges love working hardware demos.

Future Scope

- Add iris recognition with specialised camera
 - Battery backup and standalone mode on Raspberry Pi (port JavaFX)
 - Anti-spoofing liveness detection
-

#9**WattWise Auditor – Intelligent Home Energy Disaggregation & Carbon Footprint Advisor****Problem Statement**

Homeowners receive monthly electricity bills with no breakdown of appliance-wise consumption, making it hard to reduce waste and carbon footprint. Existing energy monitors require cloud analytics or dedicated hardware.

Proposed Solution

A desktop app that imports whole-house power readings (from a CSV file exported by a low-cost open-hardware monitor or manually entered) and applies Non-Intrusive Load Monitoring (NILM) algorithms offline to disaggregate the signal into individual appliance usage. It then provides personalised energy-saving recommendations and a carbon footprint estimate.

Innovation Factor

- Offline NILM using a combinatorial optimisation and hidden Markov model (HMM) implemented in Java, no cloud needed
- Appliance signature library that can be trained by the user with simple on/off events
- Environmental impact analysis using local emission factors

- Gamified savings challenge and personalised tips

Abstract (330 words)

Understanding where electricity is consumed in a home is the first step to conservation, but detailed appliance monitoring is expensive. WattWise Auditor brings sophisticated energy disaggregation to any laptop, completely offline. The user supplies a CSV of aggregate power readings sampled every few seconds (from open-source monitors like EmonPi or even a smart plug's export). The application first preprocesses the signal to detect edges (step changes), which correspond to appliances turning on/off. A built-in training mode allows the user to label these events (e.g., "fridge", "TV") to create an appliance signature database. Alternatively, a default signature library derived from public datasets (like REDD) can be bundled.

The disaggregation engine uses a Hidden Markov Model whose states represent combinations of appliance states. It estimates the most likely sequence of appliance activities that produced the observed aggregate power, utilising the Viterbi algorithm. The result is a minute-by-minute breakdown of consumption per appliance. The JavaFX dashboard visualises this as stacked area charts, pie charts, and daily timelines. The system then computes a carbon footprint using a local table of grid emission factors (customisable per region) and generates a set of actionable recommendations (e.g., "Your refrigerator consumes 30% more than similar models; check the door seal").

All processing is local, preserving privacy. The interface includes a "savings challenge" where users set reduction goals and track progress. WattWise Auditor educates and empowers users to reduce energy waste, directly contributing to environmental sustainability.

Core Features

- CSV import of aggregate power data
- Edge detection and event labelling for appliance training
- HMM-based disaggregation engine (Viterbi)
- Appliance consumption breakdown visualisation
- Carbon footprint calculation
- Personalised recommendation engine
- Goal setting and progress tracking

Functional Requirements

1. Load power data and visualise raw aggregate signal.
2. Detect events and allow manual/automatic labelling.
3. Train appliance signatures (power levels, duration distributions).

4. Run disaggregation and estimate appliance power over time.
5. Display per-appliance charts and statistics.
6. Calculate CO₂ and provide tips.
7. Save/load projects.

Non-Functional Requirements

- Performance: Disaggregation of 1-week data (1-min resolution) within 30 seconds.
- Accuracy: Achieve >80% energy assignment on known test datasets.
- Usability: Clear visualisations, simple calibration workflow.

Suggested Technologies

- JavaFX with charts
- Apache Commons Math (HMM implementation)
- SQLite (appliance signatures, project data)
- OpenCSV for data import
- JFreeChart (alternative if needed)

System Modules

1. **Data Importer** – parsing and resampling
2. **Event Detector** – edge detection algorithm
3. **Appliance Trainer** – signature creation
4. **Disaggregation Engine** – HMM Viterbi
5. **Visualisation Dashboard**
6. **Advisor** – carbon & recommendation logic

Architecture Recommendation

Pipeline with strategy for different disaggregation algorithms. MVC for UI. Use a command pattern for undoable labelling.

Database Design Overview

Tables: Project, RawData, Event, ApplianceSignature, DisaggregatedResult.

AI/ML Possibility

Yes – HMM is a statistical model; could extend to deep learning (Deeplearning4j) for sequence-to-point NILM offline.

Difficulty Level

Intermediate–Advanced

Estimated Completion Time

8 weeks (2 weeks data handling & UI, 3 weeks event detection & HMM, 2 weeks training & recommendations, 1 week testing)

Competition Winning Potential

Strong – tackles environmental sustainability with a technically deep approach; judges appreciate the blend of signal processing, machine learning, and green tech.

Future Scope

- Real-time ingestion from serial port (USB power monitor)
 - Multi-phase electrical panel support
 - Community benchmarks and anonymous comparison (offline data packs)
-

#10

DocuSanitizer Studio – Intelligent Offline Document Redaction & PII Protection

Problem Statement

Organisations handling sensitive documents (legal, medical, government) must redact personally identifiable information (PII) before sharing. Manual redaction is error-prone; automated tools often require cloud OCR and AI, risking data leaks.

Proposed Solution

A desktop application that processes scanned or digital documents, performs OCR offline (Tesseract), then uses NLP and pattern matching to detect PII (names, IDs, phone numbers, addresses). It automatically suggests redaction boxes, allows user review, and outputs a sanitised PDF with permanent redaction—all without internet.

Innovation Factor

- End-to-end offline redaction pipeline: OCR → NER → redaction application
- Customisable PII detection rules using regex and a local NLP model (OpenNLP name finder)
- Visual review interface showing detected entities with confidence scores
- Creates redacted PDFs with proper permanent removal (not just overlay)

- Fully air-gapped ready, addressing a critical privacy compliance gap

Abstract (330 words)

Data breaches often result from overlooked PII in shared documents. DocuSanitizer Studio is a privacy-first, offline redaction tool for sensitive documents. The user imports a PDF or set of scanned images; the application performs OCR using Tesseract with bundled language packs, producing searchable text. It then runs a multi-layer PII detection engine: first, a fast regex-based scanner catches structured identifiers (SSN, credit card numbers, phone numbers, email patterns). Next, a named entity recognition (NER) model from Apache OpenNLP identifies personal names, organisations, and locations. The system also applies custom dictionary matching (e.g., employee ID prefixes). Each detected entity is assigned a confidence score and highlighted on the document image with a bounding box.

A dedicated review interface lets the user inspect each suggestion, adjust boundaries, and mark additional areas manually. Once approved, the tool renders a new PDF where the selected regions are permanently removed (rasterised black boxes or pixel-level deletion), ensuring no underlying text can be recovered. The application works entirely offline, so the documents never leave the local machine, making it suitable for legal firms, clinics, and government offices with strict data handling policies. DocuSanitizer Studio combines OCR, NLP, and secure PDF manipulation into a practical, compliance-oriented solution that stands out for its focus on privacy and security.

Core Features

- PDF/Image import and offline OCR (Tesseract)
- Multi-engine PII detection (regex, NER, dictionary)
- Interactive review and manual redaction tools
- Permanent redaction rendering (rasterised black boxes)
- Export as clean redacted PDF
- Redaction audit log (what was redacted, by what rule)

Functional Requirements

1. Load multi-page PDF/images and run OCR.
2. Detect PII using configurable rule sets.
3. Display bounding boxes with labels and confidence.
4. Allow user to accept/reject/edit redaction regions.
5. Generate final redacted PDF with permanent removal.
6. Save project for later resumption.

Non-Functional Requirements

- Security: Zero network calls; temporary OCR images securely deleted.
- Performance: Process a 10-page document under 2 minutes.
- Usability: Intuitive review panel with keyboard shortcuts for fast redaction approval.

Suggested Technologies

- JavaFX (UI and PDF/image viewing)
- Tesseract OCR (via JavaCPP)
- Apache OpenNLP (NER models for English, etc.)
- Apache PDFBox (PDF manipulation and redaction)
- SQLite (project state)

System Modules

1. **Document Loader** – PDF/Image parsing
2. **OCR Engine**
3. **PII Detection Engine** – regex + NER
4. **Review Workspace** – visual annotation editor
5. **Redaction Renderer** – PDFBox based permanent removal
6. **Audit Logger**

Architecture Recommendation

Pipeline: Document → OCR → Text → Detection → Annotations → User Review → Render. Use command pattern for editing actions.

Database Design Overview

Tables: Project, Page, DetectedEntity, RedactionRegion. Text coordinates stored for mapping.

AI/ML Possibility

Yes – NER model from OpenNLP is a statistical model; could add offline custom model training on domain-specific data using OpenNLP's API.

Difficulty Level

Intermediate

Estimated Completion Time

8 weeks (2 weeks OCR integration, 3 weeks detection pipeline, 2 weeks review UI & redaction rendering, 1 week packaging)

Competition Winning Potential

Very high – privacy and security are hot topics; a fully offline, practical document sanitisation tool with immediate applicability in many sectors will impress judges.

Future Scope

- Add image redaction (face blurring) using OpenCV
- Redaction of audio/video metadata
- Batch processing of entire folder with pre-configured rules